

Del copiar y pegar al agente que opera archivos

Qué cambió técnicamente para que un modelo de lenguaje pueda leer código y modificar archivos locales — y en qué se diferencia de solo "llamar a una API".

Hasta hace poco, usar IA era copiar y pegar texto en un navegador.

Hoy, un modelo se conecta a tu entorno de desarrollo, lee código y modifica archivos locales.

¿Qué cambió técnicamente para que eso sea posible — y en qué se diferencia de simplemente “llamar a una API”?

Un LLM, solo, no puede tocar archivos

Es texto-entra / texto-sale: no tiene llamadas al sistema operativo. Dos tipos de razones lo explican.

ARQUITECTURA

Sin canal al disco

- El modelo corre en un servidor remoto, sin acceso al disco del usuario.
- Ni siquiera corriendo localmente tiene primitivas de entrada/salida.

SEGURIDAD

Aislamiento deliberado

- Se necesita consentimiento explícito del usuario.
- Riesgo de inyección de instrucciones si el modelo actuara libre sobre archivos.

MCP vs. API — el concepto central

API TRADICIONAL

Contrato entre programas

La persona desarrolladora decide, **de antemano y en el código**, qué endpoint se llama y cuándo. El programa no decide nada en tiempo de ejecución.

VS

MCP

Protocolo abierto, JSON-RPC 2.0

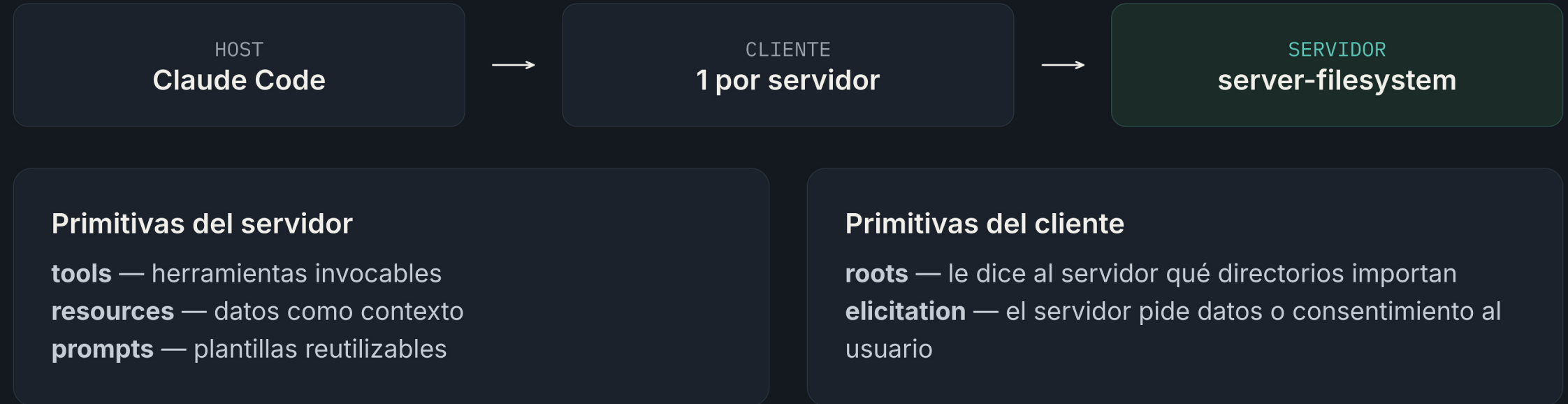
Un servidor publica un catálogo de herramientas. El cliente lo descubre **en tiempo de ejecución**, y es **el modelo** quien decide cuál invocar según lo que pide el usuario.

La comparación completa

Dimensión	API tradicional	MCP
Quién decide qué se invoca	La persona desarrolladora, de antemano	El modelo, en tiempo de ejecución
Descubrimiento de capacidades	Documentación externa, leída antes de programar	Catálogo descubierto en tiempo de ejecución
Acoplamiento cliente-servicio	Alto: código atado a esa API	Bajo: cualquier cliente habla con cualquier servidor
Formato de mensajes	Varía: REST/JSON, XML, gRPC...	Estandarizado: JSON-RPC 2.0
Autenticación y consentimiento	Definida por cada proveedor	Contemplada en el protocolo (OAuth remoto, elicitation)
Reutilización entre apps	Baja: cada app reimplementa su integración	Alta: un mismo servidor sirve a múltiples clientes

MCP no sustituye a las APIs: casi siempre envuelve una API o recurso existente y agrega una capa que lo hace descubrible e invocable por un modelo.

Arquitectura de MCP



Transportes: **stdio** (servidor local, subprocesso) y **Streamable HTTP** (servidor remoto).

El servidor de sistema de archivos

"FS" **no es parte del protocolo MCP** — es uno de los servidores de referencia, entre muchos posibles.

`list_directory`

`read_text_file`

`write_file`

`edit_file`

`search_files`

`move_file`

`list_allowed_directories`

El alcance se delimita con **directorios permitidos**, indicados al arrancar el servidor (por argumento) o vía el protocolo de **Roots**.

Demo en vivo

Cliente: Claude Code · Servidor: @modelcontextprotocol/server-filesystem · Directorio: workspace-demo/

01

Listar

el directorio

02

Leer

un archivo existente

03

Crear

un archivo nuevo

04

Modificar

un archivo existente

05

Buscar

un archivo por
nombre

06

Intento fuera

rechazado

Hallazgo en vivo: la primitiva Roots

Al conectar el servidor, `list_allowed_directories` reveló que el directorio realmente autorizado era **todo el proyecto**, no solo `workspace-demo/`.

Causa: Claude Code implementa el protocolo de **Roots** y comparte automáticamente la raíz del proyecto con el servidor — esa notificación **reemplaza** el argumento de arranque, no lo restringe más.

Seguridad

RIESGOS

- Inyección de instrucciones vía contenido de archivo
- Acceso a rutas fuera del directorio autorizado
- Escritura o borrado no deseados

MITIGACIONES

- Confirmación humana antes de ejecutar
- Alcance limitado a un directorio, permisos de solo lectura cuando aplica
- Revisión del catálogo de herramientas antes de conectar cualquier servidor

PRUEBA DEL LÍMITE

Listar /home/jesus/Desktop/Moviles → rechazado: "Access denied - path outside allowed directories". Lo impidió la validación de rutas del propio servidor MCP, no un permiso del sistema operativo.

Casos de uso reales

Claude Code

Terminal agéntico con soporte nativo de MCP

Cursor

Editor de código con integración MCP

Google Antigravity

Plataforma agéntica de Google (no confundir con Qwen, que es solo una familia de modelos)

Las tres comparten el mismo patrón: el host tiene acceso directo y local al proyecto vía un servidor MCP de sistema de archivos, así que el modelo lee y escribe archivos reales sin que la persona copie y pegue código manualmente.